

In this assignment you will use MATLAB/Python to explore ideas from class. **Important:** please read the submission instructions posted on Canvas before you begin, as well as the coding tutorial. **Remember: The only parts you need to include in your write-up are those marked with a star (*)!**

The gradient

1 a* For a smooth function of a single variable, $f(x)$, the derivative at a point x_0 , i.e. $f'(x_0)$, is a scalar that gives the slope of the tangent line to the curve at the point (x_0) . For a smooth function of two variables, $f(x, y)$, how many tangent lines are possible at any given point (x_0, y_0) in its domain?

b* Consider a surface whose points are given by $(x, y, f(x, y))$. How is the gradient computed? Is it a scalar, a vector in 2d, or a vector in 3d? How do you plot it in $\mathbb{R}^3$ (i.e., what are its $\mathbb{R}^3$ coordinates)?

c* How is the gradient related to the directional derivative?

d* Suppose you are climbing up a mountain. If you deviate from the gradient's direction, why does your path to the top necessarily have to be longer?

A. coli

MATLAB: Make sure you have `acoli_hist.m` in your working directory.

Python: Make sure you have `acoli.py` in your working directory, then import the `acoli_hist` function with: `from acoli import acoli_hist`.

Please read the function documentation carefully to learn how it should be used. In MATLAB, do `help acoli_hist`; in a Python, that will depend on your IDE, but you can always simply open the script file and read it.

Note that the `acoli_hist` function returns multiple outputs (the histogram and the ground truth food function). To get both, you will have to supply two output variables. In other words, your call to `acoli_hist` should resemble `[histm, food] = acoli_hist(...)`.

2 a In your simulations, *A. coli* will move through an environment where the food concentration is fixed over time and is a smooth function of the position coordinates; we can write it as $f = (x, y)$. Visualize food function 1¹ in three different ways: as a heat map using `imagesc` (in Python, you need to use `imshow` with the argument `origin='lower'` because this is not an actual image); as a surface plot using `surf` (`plot_surface` in Python, see tutorial); and as a contour plot using `contour`. Then, do the same for food function 2. Observe the main differences between the two functions. Also, be sure to check the effect of selecting different values for the number of bins.

— **b** Now display the gradient vector field for food functions 1 and 2 using the `quiver` function (make sure you understand how to use it first—see [this](#)). If the vectors get too cluttered, using fewer bins might give a better visualization. You are allowed to use the `gradient` function to calculate the (numerical) gradient, but be sure you understand what it does! (Please do **not** use `imgradient`.) Note that the inputs to `quiver` will be the outputs from the `gradient` function – make sure you understand what they mean. Also, remember that the x-axis is the horizontal axis (meaning columns, i.e. the *second* axis), whereas the y-axis is vertical (meaning rows, i.e. the *first* axis)!

Finally, **check that the vector field you get is what you expected**. This is how you will know your procedure is correct for later use in **3c**.

— **c** Using food function 1 (you can pick the starting location), experiment with different values for the run-to-tumble (`rttp`) and tumble-to-run (`ttrp`) transition probabilities. Make sure you get a feeling for how they affect *A. coli*'s behavior. In particular,

¹That is, visualize the food matrix that is returned by the `acoli_hist` function. This matrix encodes the food function value at every (x,y) position within the specified range. (There's no need to inspect the source code for that!)

- what happens when both are low?
- both are high?
- one is high, the other low?
- either one of the probabilities is 0?
- how many steps are typically necessary for A. coli to approach the food source?

To see the behavior of A. coli during the simulation, set the `visualize` parameter to `true`² (just be sure to select a small number trials to visualize or the movie will play for a very long time!). Note that if you want to abort a long-running command, you can use `ctrl+c`. Remember to close the animation window afterwards.

– **d** For some choice of parameters, use `acoli_hist` to obtain histograms with 10, 50, 100 and 500 bins along each axis. Visualize them and observe the relationship between the number of bins used and the resulting histogram. (Since each histogram is a 2d array, you can use `imagesc/plt.imshow` to plot them.) Are more bins always better?

– **e*** Given your observations from part **c**, what are good `rttp` and `ttrp` values to pick if your goal is to maximize the average value of the food function seen by A. coli over a long period of time? (Hint: You can calculate this average easily using the `histm` and `food` matrices.³) Explain why this combination is effective. To verify that you made a good choice, **build a table** with the resulting food consumption for several (`rttp,ttrp`) pairs of values between 0 and 1). Please use 50 bins, and make sure you run at least 100 trials with 200 steps per trial (this may take a while, so it is a good idea to turn the `verbose` option off!).

– **f*** Using an initial position of (0,0) for A. coli, try running the simulation several times

²in Python, the animation won't play correctly if you run it inside a notebook.

³For the purposes of calculating the average, you may either pretend the food function is zero outside the area covered by the `food` and `histm` matrices (underestimating the true average), or simply ignore any steps that occur outside this area (overestimating the true average).

on food function 2, using the values chosen in part [e](#). Could different values do better for food function 2? Why or why not? If you find values you think are better, confirm this empirically by calculating the average value of the food function seen by *A. coli* for both the old values from [e](#) and your new values (again, use 50 bins, and at least 100 trials with 200 steps per trial). Note: building a table here will also be helpful.

[g*](#) Compare and contrast *A. coli*'s strategy with the general gradient ascent algorithm (see lecture notes). Are there any advantages to the biologically-inspired version?

Image manipulation: the image as a matrix

[3a](#) Open the image `Paolina.tiff` and *convert it to a matrix with double precision numbers* (please refer to the tutorial for how to do this). Display the image in grayscale and using any other colormap of your choice.

[b*](#) Plot the image level sets using the `contour` function. You will need to adjust the axes by typing `axis ij` in MATLAB—this is the image coordinate axis in which the y-axis increases from top to bottom (to do the equivalent in Python, pass the additional argument `origin='upper'` to `contour` and set `plt.axis('image')`). Comment briefly on what you see. What does the `contour` function do?

[c*](#) Display the gradient vector field (as in [1b](#)) for the area of the image around Paolina's face by taking the slice `(150:300, 50:150)` (adjust the axis as in [b](#)⁴). You might want to use the zoom tool in the figure window to zoom in on the vector field around the nose until you can see the direction of individual vectors. Explain briefly why the length and directions of the vector are what they are in this region.

[d*](#) Compute the image gradient magnitudes (lengths of the gradient vectors) at each

⁴in Python, you might need to additionally flip the image along the 0-axis using `np.flip`.

image location as the matrix `gmag`. The magnitude (length) of a vector $\mathbf{v}$ is given by the formula

$$\|\mathbf{v}\| = \sqrt{v_x^2 + v_y^2}.$$

Display `gmag` in color code with `imagesc/plt.imshow`, and think about what information this image conveys. What do the gradient magnitudes tell us? Note: You should compute the gradient's magnitudes yourself; do **not** use the `imggradient` function.

- **e*** What would happen if we used the Paolina image as the food function for *A. coli*? What would determine the amount of “food” in this case? Choose an initial position for *A. coli* and *describe* what you expect its trajectory to be like, if you used the optimal parameters found in part 2 (no need to write code here).